



L4

模型微调与 私有化大模型

9.大模型部署

课程定位



你怎么把训练好的大模型变成业务里能赚钱的服务。

懂部署的人,才真正掌握了 LLM 的商业化。

学完能做到

- 1 计算模型在显卡上的并发上限
- 2 解释 vLLM 为什么比 transformers 快几十倍
- 3 独立部署 vLLM 并跑出性能曲线
- 4 从业务目标反推硬件采购方案

课程总览



4 个模块,环环相扣,每个模块都有一个「带走能用」的核心产出

1

显存计算

核心产出

心算法 + 显存预算

2

推理框架

核心产出

理解 vLLM 为什么快

3

部署性能

核心产出

Rooftop 性能曲线

4

显存规划

核心产出

业务→硬件方案

01

模块一

显存计算

从「显存」到「能扛多少并发」

6 个知识点 · 1 个心算法

引子:老板的三个问题

老板说:“我们要把客服系统从 OpenAI API 切到本地 7B 模型,你给我个方案。”

你必须回答:

Q

我们要买什么卡?

硬件选型

Q

一台机器能服务多少同时在线用户?

容量规划

Q

用户体验和原来比会怎样?

性能预期

五大核心指标

TTFT	首 token 延迟	Prefill 阶段决定	用户敏感
TPOT	每 token 间隔	Decode 阶段决定	流畅度
E2E	端到端总延迟	$TTFT + TPOT \times \text{输出长度}$	总耗时
Throughput	吞吐(tokens/s)	系统总产出能力	老板关心
并发上限	满足 SLA 的并发数	容量边界	业务上限

延迟是用户的事,吞吐是老板的事 —— 你的工作是在两者之间画一条曲线

模型大小换算:B ≠ GB

核心公式

模型大小 = 参数量 × 精度字节

7B × 2字节(FP16) = 14 GB

⚠ 学生最容易踩的坑

"7B = 7GB"是错的!

B 是参数量(Billion),GB 是存储,中间隔着「精度字节数」。只有 INT8/FP8 时数值才恰好相等。

精度阶梯(以 7B 模型为例)

FP32	4 字节		28 GB
FP16 / BF16	2 字节		14 GB
FP8 / INT8	1 字节		7 GB
INT4 (AWQ)	0.5 字节		3.5 GB

KV Cache:第二大显存消耗

单 token KV

$$= 2 \times L \times H_{kv} \times d_{head} \times dtype \text{字节}$$

参数来源 • config.json

L

层数

num_hidden_layers

H_{kv}

KV head 数

num_key_value_heads

d_{head}

head 维度

hidden_size /
num_attention_heads

常见模型单 token KV(FP16)

Qwen2.5-7B

56 KB

Qwen2.5-14B

192 KB

Qwen2.5-72B

320 KB

LLaMA-3-70B

320 KB

显存预算速查公式

两步公式

① KV Cache 可用空间

- = 总显存 $\times 0.9$
- 模型权重(参数 $\times$ dtype 字节)
- 2 GB(运行开销 R)

② 能支持的并发请求数

- = KV Cache 可用空间
- $\div$ (单 token KV $\times$ 上下文长度)

6 步心算法

- ① 显存多少?
- ② 乘 0.9
- ③ 减权重(参数 $\times$ dtype)
- ④ 再减 2 GB
- ⑤ 单 token KV $\times$ 上下文
- ⑥ ④ $\div$ ⑤ = 并发数

口诀:九折拿余量,扣掉两胖子(权重 + 2GB),剩下分用户

完整算例:4090 + Qwen2.5-7B + 4k 上下文

第一步:KV Cache 可用空间

总显存	24 GB
× 0.9 利用率	21.6 GB ← vLLM 实际可用
- 权重(7×2)	-14 GB ← FP16
- 运行开销 R	-2 GB ← 7B 经验值

KV Cache 池 **5.6 GB**

第二步:能支持的并发请求数

单 token KV(7B)	56 KB
× 上下文 4096	≈ 0.224 GB/请求

$$5.6 \div 0.224 =$$

25 并发

理论上限 25,实际舒适 18~20

注意:并发请求数 ≠ 用户数。客服场景 think time × 10,25 并发实际可服务 ~250 活跃用户(详见模块 4)

02

模块二

推理框架

为什么 vLLM 比裸 transformers 快几十倍?

3 个核心技术 · 1 张选型决策树

Continuous Batching: GPU slot 不空闲

Static Batching(传统)

A:	A1	A2	A3			
B:	B1	B2	B3	B4	B5	B6
C:	C1	C2				

△ slot 闲置,新请求等待

Continuous Batching(现代)

A:	A1	A2	A3	D1	D2	D3
B:	B1	B2	B3	B4	B5	B6
C:	C1	C2	E1	E2	E3	E4

✓ slot 永不空闲,完成立即补位

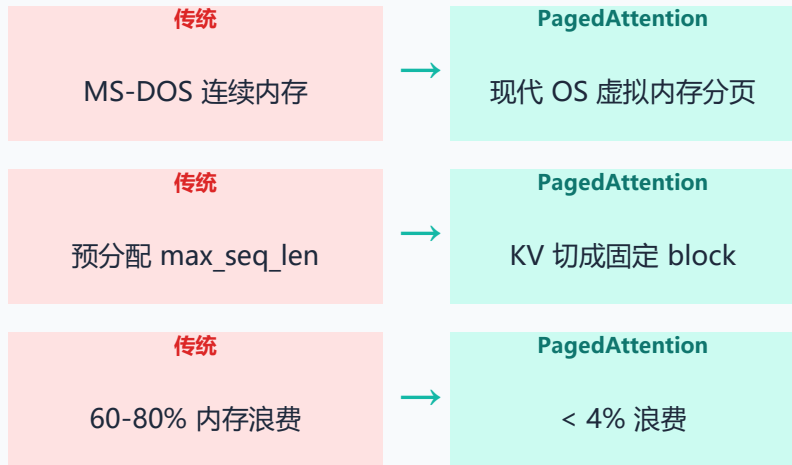
核心理想

批处理粒度从「请求级」降到「迭代级」

收益: GPU 利用率 30-40% → 80-90%, 吞吐 2-4x

PagedAttention: KV Cache 的虚拟内存

操作系统类比



做法

- 1 KV Cache 切成固定大小 block
比如每 16 token / block
- 2 每个请求维护「块表」
类似 OS 页表
- 3 按需申请, 不预分配
用一段算一段
- 4 释放的 block 可被新请求复用
消除外部碎片

收益: 显存浪费 60-80% → < 4% · 同硬件并发能力 2-4x

Prefix Caching + 引擎选型

🎁 Prefix Caching · RAG 与长系统提示的免费午餐

共享前缀的 KV 只算一次。系统提示 2000 token + 用户问 50 token → prefill 提速 ~40x

启用方法: vLLM 启动加 `--enable-prefix-caching` 即可

主流引擎选型

默认

vLLM

生态最强,通用首选

长前缀

SGLang

RadixAttention, Agent / RAG

本地

Ollama

桌面、原型、教学

国产

LMDeploy

昇腾、InternLM

"vLLM 优化的是装得下, SGLang 优化的是不重复算"

vLLM:通用 GPU 推理的事实标准

🏆 没有特殊需求,就选它 —— 业界默认安全选项,生态飞轮已经转起来

出身 · 核心技术

团队	UC Berkeley Sky Lab (2023)
核心论文	PagedAttention (SOSP 2023)
核心技术	PagedAttention + Continuous Batching
后续追加	Prefix Caching、Chunked Prefill、 Multi-LoRA、Speculative Decoding、PD 分离

一行命令起服务

```
$ vllm serve Qwen/Qwen2.5-7B-Instruct \
  --max-model-len 8192 \
  --enable-prefix-caching \
  --gpu-memory-utilization 0.9
```

👉 自动暴露 OpenAI 兼容 API (默认 8000 端口)

- ✓ 模型 day-1 适配最快(新模型几乎首发即支持)
- ✓ 生态最丰富、文档最全、中文资料多
- ✓ OpenAI 兼容 API,切换零成本
- △ 极致长前缀复用场景不如 SGLang

SGLang:Agent 与长上下文的新王

⚡ RAG / 多轮 / Agent / DeepSeek-V3 — 这些场景下,它比 vLLM 快出一个量级

出身 · 核心技术

团队	LMSYS (Vicuna、Chatbot Arena 作者)
起点	2024,从结构化生成 DSL 出发
核心技术	RadixAttention(前缀树自动复用)
加分项	原生 xgrammar 结构化输出
代表案例	DeepSeek-V3/R1 官方参考实现

一行命令起服务

```
$ python -m sglang.launch_server \  
  --model-path Qwen/Qwen2.5-7B-Instruct \  
  --port 30000
```

👉 RadixAttention 默认开启,无需配置

👉 同样提供 OpenAI 兼容 API

- ✓ 长共享前缀场景(RAG/多轮)碾压 vLLM
- ✓ 结构化输出最强(JSON Schema / 正则约束)
- ✓ DeepSeek-V3/R1、大型 MoE 模型部署首选
- △ 新模型 day-1 适配通常晚 vLLM 1-2 周

03

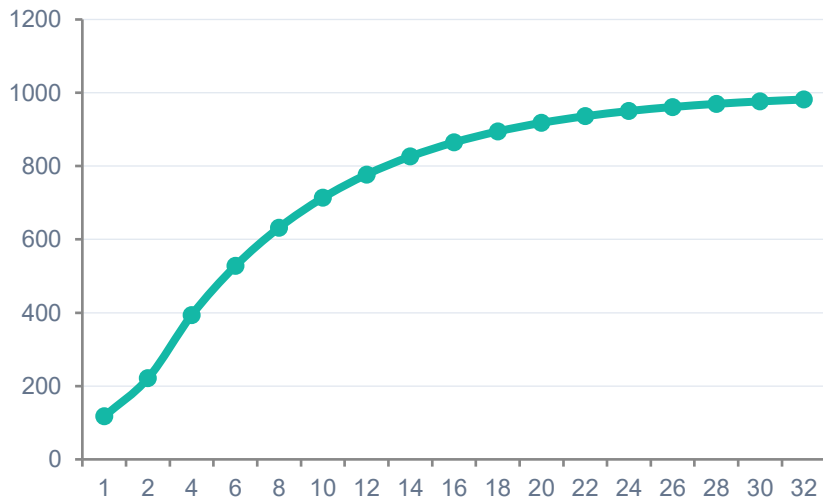
模块三 · 实操核心

部署性能

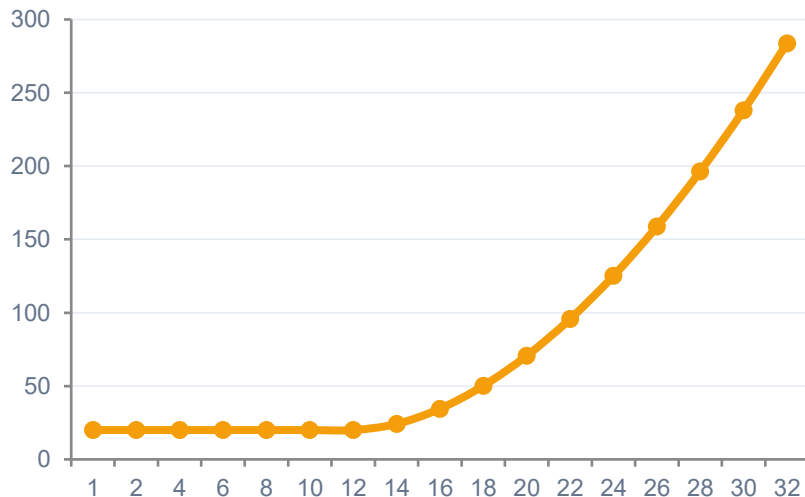
带走一张自己跑出来的性能曲线

核心张力:吞吐 vs 延迟 Rooftop 曲线

吞吐 vs 并发(饱和)



TPOT vs 并发(拐点)



找到拐点 = 在拐点附近运营 = 工程师真正的工作

启动服务 + 压力测试

① 启动 vLLM 服务

```
$ vllm serve Qwen/Qwen2.5-7B-Instruct \  
  --max-model-len 8192 \  
  --enable-prefix-caching \  
  --gpu-memory-utilization 0.9
```

② 扫并发跑 vllm bench

```
for C in 1 2 4 8 16 32; do  
  vllm bench serve --model Qwen/Qwen2.5-7B-Instruct \  
    --random-input-len 1024 --random-output-len 128 \  
    --num-prompts $((C * 20)) --max-concurrency $C  
done
```

产出:三张性能曲线图

1

吞吐 vs 并发

饱和曲线

教学价值

看 GPU 在哪里"装满"

2

TPOT P95 vs 并发

拐点曲线

教学价值

看延迟在哪里"崩"

3

吞吐 vs TPOT

Rooftop 图

教学价值

找黄金运营点

学生离开课堂时,必须带走自己跑出来的这三张图

五大压测陷阱

1 没 warmup

✗ 第一批冷启动数据

✓ 先丢 10-20 个请求热身

2 流式没开

✗ TTFT 全算成 E2E

✓ 使用流式 API

3 Prompt 太短

✗ 只测出 decode 上限

✓ 用真实业务长度

4 Prompt 全相同

✗ Prefix cache 假象

✓ 用 random 数据集

5 只看 P50

✗ 1% 用户已崩你不知

✓ 必看 P95、P99

P50 给自己看, P95 给用户看, P99 给老板擦屁股

04

模块四

显存规划

从业务目标反推硬件采购方案

GPU 选型 · Think Time 倍数 · 生产化清单

GPU 选型速查

RTX 4090

24 GB · 消费级

7B 舒适 / 14B 量化

¥1.4-1.8万

L40S

48 GB · 专业卡

30B 舒适 / 70B 量化

企业级

A100

80 GB · 数据中心

70B 主力

¥10-15万

H100 / H200

80-141 GB

70B+ / 训练推理双修

¥20-30万

昇腾 910B

64 GB · 国产

国产化部署事实标准

受限

"7B 选 4090, 30B 选 L40S, 70B 选 A100/H100, 671B 别自己部署"

选型四大反直觉



推理是带宽瓶颈

不是算力瓶颈

Decode 阶段 GPU 大部分时间在等显存读权重。所以 H20 算力被砍 85%,推理仍很强;Mac M3 Ultra 推理快(819 GB/s 带宽)



多张 4090 $\neq$ 一张 A100

通信差几个量级

4090 没有 NVLink,多卡只能走 PCIe(64 GB/s)。A100/H100 之间 NVLink 高达 600-900 GB/s。70B+ TP 部署一张 A100 远胜多张 4090



显存大 $\neq$ 推理快

两个独立维度

24 GB 4090 装得下 14B-AWQ,但速度可能不如 80 GB A100。显存决定能不能装,带宽决定快不快



H20 反直觉

中国市场另类首选

算力只有 H100 的 15%,但带宽是 H100 的 1.2 倍。训练几乎不能用,推理出乎意料地强 —— 这就是为什么国内大厂买它做推理

并发请求数 $\neq$ 用户数

活跃用户数 $\approx$ 并发请求数 $\times$ Think Time 倍数

Think Time 倍数 = 用户思考间隔 $\div$ 单次请求耗时

场景	单次耗时	思考间隔	倍数	25 并发对应
客服机器人	3 秒	30 秒	10x	~250 用户
长文创作	20 秒	120 秒	6x	~150 用户
代码补全	1 秒	5 秒	5x	~125 用户
Agent 循环	10 秒	0 秒	1x	~25 用户

💡 Agent 没有 think time $\rightarrow$ 用户数 $\approx$ 并发数 $\rightarrow$ 这就是为什么 Agent 应用贵

从业务到硬件 + 生产化清单

容量规划推演链路

业务目标

10000 日活

× 同时在线率 5-10%

500-1000

× 实时请求率(场景)

50-100 并发

÷ 单卡并发(25)

2-4 卡

× 余量(1.5x)

3-6 卡

生产化必备清单



容器化

Docker + GPU Container



负载均衡

Nginx / LiteLLM 多实例



监控告警

Prometheus + Grafana



鉴权限流

API Key + 配额管理



内容安全

输入输出双向审核

LLM 部署 · 七句话总结



- 1 B 不等于 GB,精度决定换算
- 2 显存预算:九折拿余量,扣掉两胖子,剩下分用户
- 3 vLLM 优化"装得下",SGLang 优化"不重复算"
- 4 推理是带宽瓶颈,不是算力瓶颈
- 5 batch 越大吞吐越高,但 TPOT 越差,找拐点
- 6 并发请求 $\neq$ 用户数,中间隔着 think time
- 7 P50 给自己看,P95 给用户看,P99 给老板擦屁股

懂部署的人,才真正掌握了 LLM 的商业化

实践：vLLM 性能扫描分析

目标: 量化 vLLM 部署性能,找到并发拐点

步骤:

- 1.本机部署 vLLM,运行 Qwen2.5-1.5B 模型
- 2.扫描 1 → 128 共 8 档并发,执行压测
- 3.采集 TTFT、TPOT、吞吐三组指标
- 4.绘制 Rooftop 曲线,识别拐点位置

产出:基于数据反推硬件并发上限与业务承载能力的工程方法论

性能实测报告

Qwen2.5-1.5B-Instruct

vLLM 性能扫描分析

从 8 个并发档位的实测数据,找到这台机器的「黄金运营点」

5 张图 · 完整解读 · 业务级结论

实验是怎么做的

实验参数

模型	Qwen2.5-1.5B-Instruct
推理引擎	vLLM
输入长度	1024 token
输出长度	128 token
Prefix Caching	已启用
并发扫描	1 → 2 → 4 → 8 → 16 → 32 → 64 → 128
压测工具	vllm bench serve

三个核心指标

TTFT 首 token 延迟

→ 反应快不快

TPOT 每 token 间隔

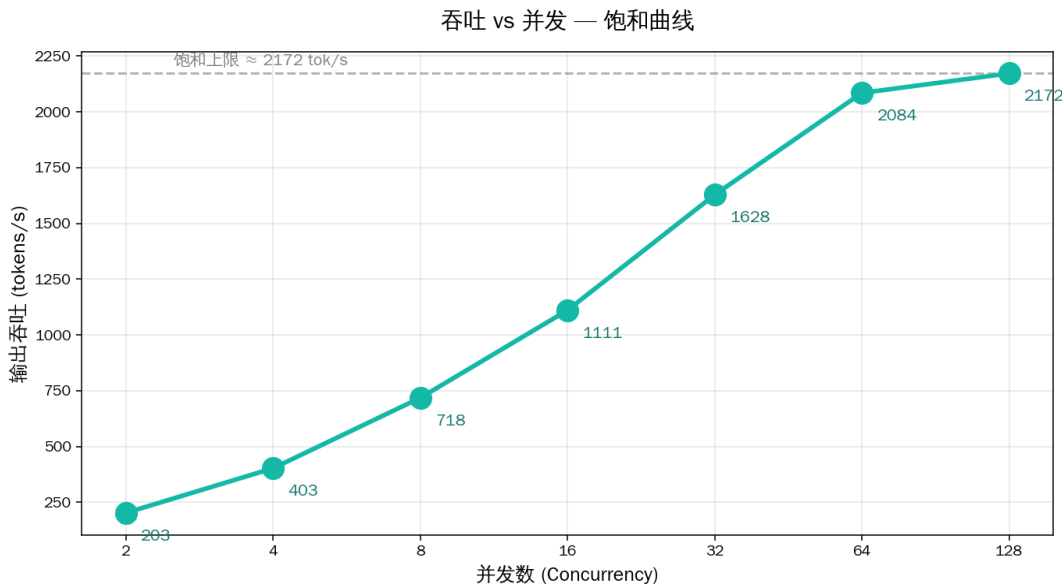
→ 回答流畅不流畅

吞吐 tokens/s

→ GPU 卖力不卖力

一句话:同时发 N 个请求,看服务的表现。 N 从 2 加到 128。

吞吐 vs 并发 — 饱和曲线



讲的是什

GPU 还能再卖力一点吗?

C=2 $\rightarrow$ 吞吐 203

C=32 $\rightarrow$ 吞吐 1628

C=128 $\rightarrow$ 吞吐 2172

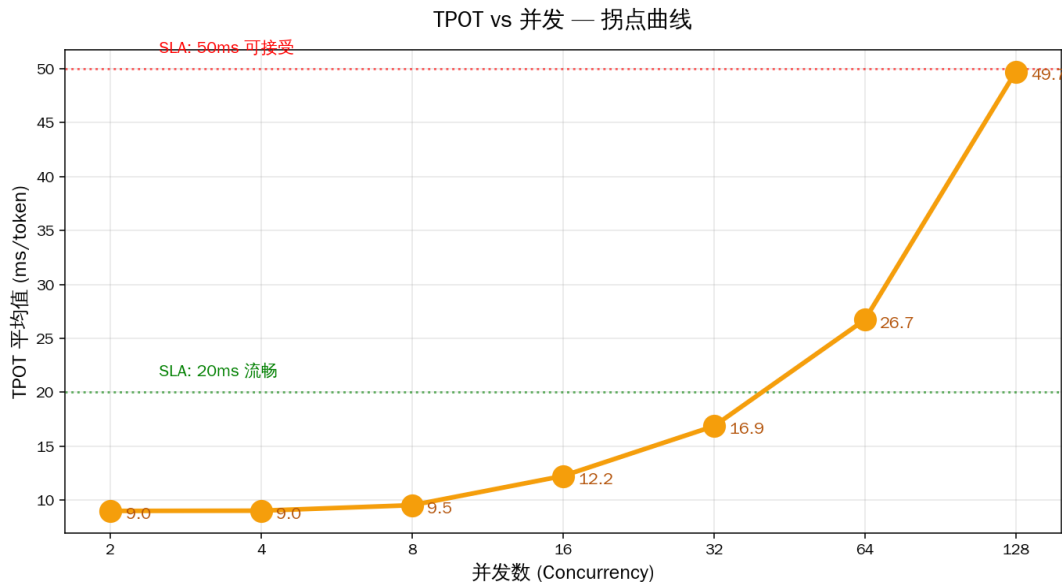
关键观察

并发翻倍但吞吐不再翻倍 —— GPU 内存和算力都是有限的,一旦满载,新请求只能排队。



这台机器跑这个模型的绝对上限 ≈ 2172 tok/s

TPOT vs 并发 — 拐点曲线



讲的是什

用户的体验有多流畅?

20ms 流畅打字

50ms 还能忍

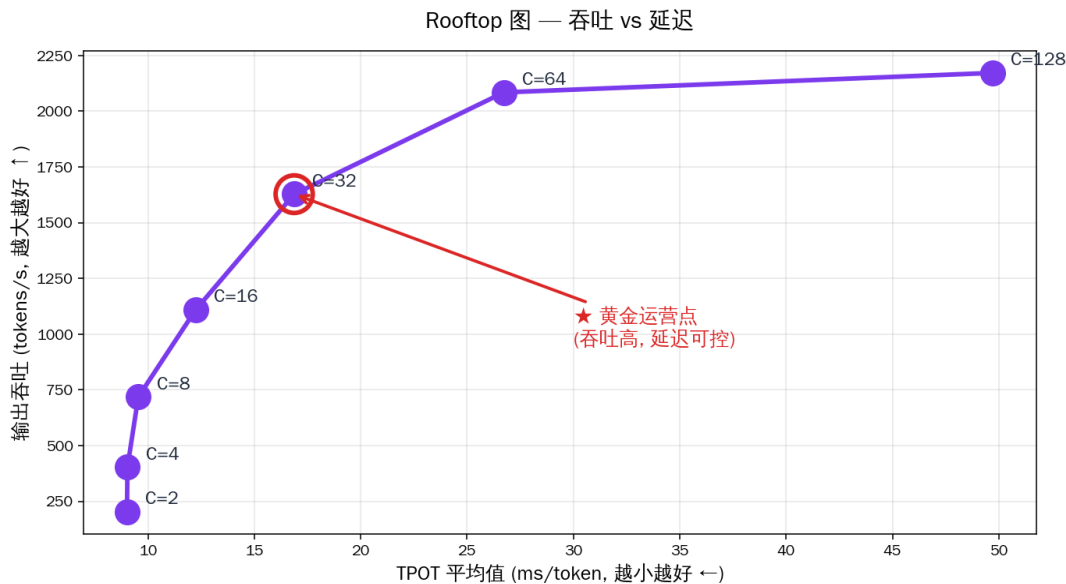
100ms+ 用户走

关键观察

曲线指数式上扬。GPU 满载后请求排队,等的人越多,每个 token 生成越慢。

💡 C=32 之后 TPOT 开始翘起,C=128 时翻到 50ms,用户能明显感觉卡顿

Rooftop 图 — 工程师真正看的图



讲的是什

延迟和吞吐怎么权衡?

横轴: TPOT (越小越好)

纵轴: 吞吐 (越大越好)

右上是天堂, 但不存在

拐点 = C=32

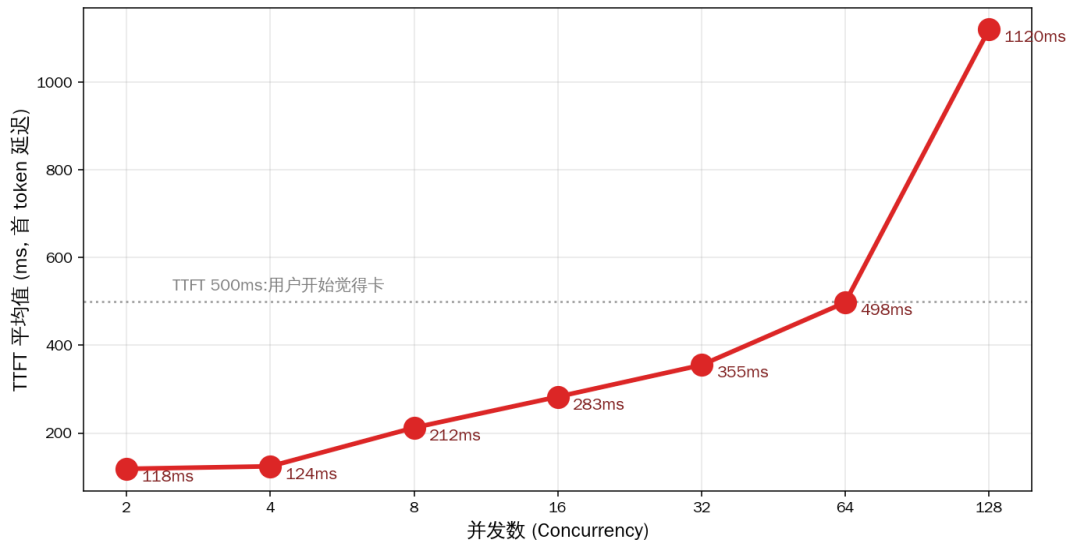
之前: 加并发主要换吞吐 (免费)

之后: 加并发主要换延迟 (亏)

★ 工程师的工作: 找到这个拐点, 在拐点附近运营

TTFT vs 并发 — 用户「第一印象」

TTFT vs 并发 — 用户感知的反应速度



讲的是什么

用户等多久才看到第一个字?

118ms 无感

498ms 觉得卡

1120ms 准备走

过载悬崖

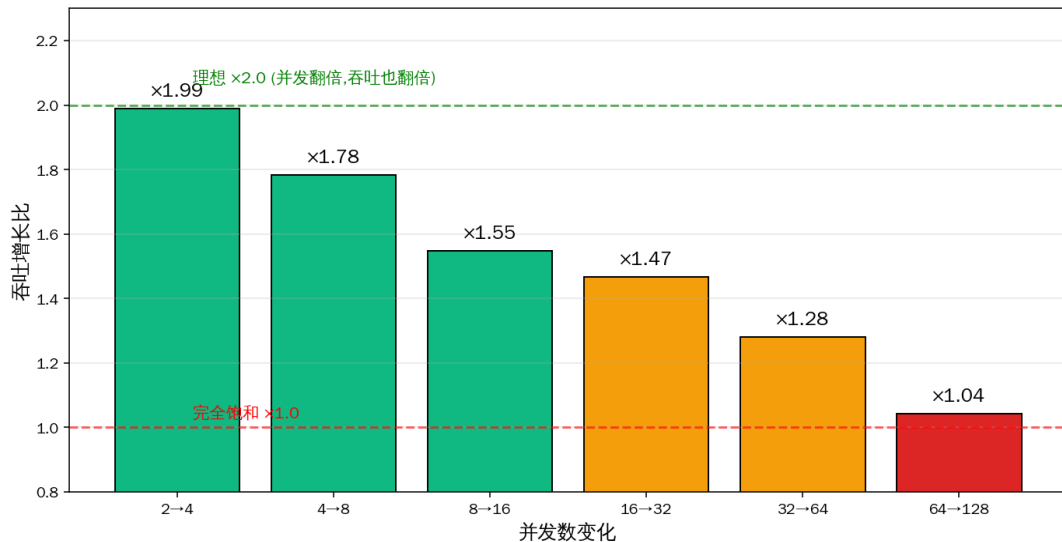
C=64→128 时 TTFT 从 498ms 跳到 1120ms, 翻 2.3 倍 —— 排队效应。



TTFT 差用户觉得反应慢, TPOT 差用户觉得说话慢, 两个都差 → 走

加并发还划算吗?

边际收益 — 加并发还划算吗?



怎么读这张图

并发翻倍 → 吞吐变成几倍?

×2.0 = 理想(免费午餐)

×1.5 = 还在赚

×1.0 = 完全白加

结论

C=64→128 时增长比 1.04 —— 翻倍并发只换来 4% 吞吐,但延迟翻倍,完全亏本。



C=64 之后再加并发完全是亏本买卖

给老板看的答案

最佳并发档

 $C \approx 32$

吞吐天花板

 $\sim 2200 \text{ tok/s}$

TPOT < 20ms 上限

 $C \approx 32$

不同业务场景的并发建议

业务场景	SLA 要求	推荐并发	能扛用户数(估算)
1 流畅档(高端)	P95 TPOT < 20ms	$C = 16$	~160 客服用户
2 推荐档(均衡)	P95 TPOT < 30ms	$C = 32$	~320 客服用户
3 极限档(高密度)	TTFT < 500ms	$C = 64$	~640 客服用户
✗ 过载档	—	$C = 128$	不推荐 · 体验差

假设客服场景: 用户思考 30 秒, 模型回答 3 秒, Think Time 倍数 $\approx 10x$

从这次实验你能学到什么

1

任何 GPU 推理服务都有 Rooftop 曲线

不管什么模型、什么显卡,形状都是普遍的

2

拐点比上限更重要

上限可以测出来,但拐点才是你实际运营的位置

3

业务 SLA 决定哪个并发能用

不同业务要看不同指标(TTFT?TPOT?吞吐?)

4

过载比想象的更可怕

C=64 $\rightarrow$ C=128 时 TTFT 飙 2.3 倍,这种悬崖效应不实测看不到

5

测了才知道

同样的模型、不同硬件,曲线形状和数字都不一样

"测不出来的优化等于没优化"

Thank You

Q & A